

第2章 关系数据库

2.4 关系代数 (扩充运算)

(2026 春季)



review: 关系代数的基本运算

关系运算		运算符	操作符
基本运算	并	$R \cup S$	$R \text{ union } S$
	差	$R - S$	$R \text{ except } S$
	笛卡尔积	$R \times S$	$R \text{ product } S$
	选择	$\sigma_F(R)$	$R \text{ where } F$
	投影	$\pi_{A_1, A_2, \dots, A_n}(R)$	$R[A_1, A_2, \dots, A_n]$

❑ 在关系代数中，并、差、笛卡尔积、选择、投影等五个运算符是基本运算，其他运算符都是扩充运算。

❑ 关系完备系统

- 采用关系数据结构，并且支持并、差、笛卡尔积、选择、投影等五个关系代数的基本运算符的系统，被称为是关系完备的系统。
- 一个关系完备的系统，能够支持所有的关系操作。

关系代数的扩充运算

- 在关系代数中，除了五个基本运算外，还引入了其他扩充运算，方便对关系操作的表示。

关系运算		运算符	操作符
扩充运算	交	$R \cap S$	$R \text{ intersect } S$
	θ -连接	$R \bowtie_F S$	$(R \text{ product } S) \text{ where } F$
	自然连接	$R \bowtie S$	$R \text{ join } S$
	外连接	$R \Join S$	$R \text{ outer join } S$
	左外连接	$R \Joinleft S$	$R \text{ left outer join } S$
	右外连接	$R \Joinright S$	$R \text{ right outer join } S$
	除	$R \div S$	$R \text{ divideby } S$

一. 关系代数的扩充运算

二. 除运算的推导

三. 扩充运算的应用

四. 关系代数小结

一、关系代数的扩充运算

□ ‘交’运算

- 运算符定义，推导公式

□ ‘连接’运算

- θ -连接的定义、推导公式
- 自然连接的定义、推导公式、如何正确使用自然连接运算
- 一种特殊的连接运算应用场景：关系的自连接(self-join)
- 外连接

□ ‘除’运算

- 运算符的定义
- 补充定义（扩展阅读）
- 为什么引入除运算
- 如何正确使用除运算

□ 交 (Intersection)

➤ R 和 S

- 具有相同的目 n
- 相应的属性取自同一个域

➤ $R \cap S$ 的运算结果

- 仍为 n 目关系，由既属于 R 又属于 S 的元组组成

$$R \cap S = \{ t \mid t \in R \wedge t \in S \}$$

➤ 交运算 满足 交换律 和 结合律

- 交换律: $R \cap S = S \cap R$
- 结合律: $(R \cap S) \cap T = R \cap (S \cap T)$

交 2

□ the set of tuples in the relation $T = R \cap S$

```
 $T := \{ \}$ 
```

```
for each tuple  $t \in R$ 
```

```
{
```

```
    /* 由既属于关系  $R$ 、又属于关系  $S$  的元组组成结果关系  $T$  */
```

```
    if ( $t \in S$ )
```

```
        then add tuple  $t$  into  $T$ 
```

```
}
```

```
return  $T$ 
```

□ 交不是一个基本运算符，其功能可以用差运算来实现

$$R \cap S = R - (R - S) = S - (S - R)$$

交 3

□ 交不是一个基本运算符，其功能可以用差运算来实现

$$R \cap S = R - (R - S) = S - (S - R)$$

R	A	B	C
	<i>a</i> ₁	<i>b</i> ₁	<i>c</i> ₁
	<i>a</i> ₁	<i>b</i> ₂	<i>c</i> ₂
	<i>a</i> ₂	<i>b</i> ₂	<i>c</i> ₁

S	A	B	C
	<i>a</i> ₁	<i>b</i> ₂	<i>c</i> ₂
	<i>a</i> ₁	<i>b</i> ₃	<i>c</i> ₂
	<i>a</i> ₂	<i>b</i> ₂	<i>c</i> ₁

R - S	A	B	C
	<i>a</i> ₁	<i>b</i> ₁	<i>c</i> ₁

S - R	A	B	C
	<i>a</i> ₁	<i>b</i> ₃	<i>c</i> ₂

R ∩ S	A	B	C
	<i>a</i> ₁	<i>b</i> ₂	<i>c</i> ₂
	<i>a</i> ₂	<i>b</i> ₂	<i>c</i> ₁

❑ 例2.10：查询至少选修1号课程和3号课程的学生号码。

$$\pi_{sno}(\sigma_{cno='1'}(SC)) \cap \pi_{sno}(\sigma_{cno='3'}(SC))$$

选修关系 SC

Sno	Cno	Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

$$\pi_{sno}(\sigma_{cno='1'}(SC)) \cap \pi_{sno}(\sigma_{cno='3'}(SC))$$

查询结果

Sno
201215121

❑ 思考：能否将例2.10表示为 $\pi_{sno}(\sigma_{cno='1'}(SC) \cap \sigma_{cno='3'}(SC))$?

➤ 不能。因为该表达式的查询结果永远为空。

运算	运算前提	结果关系模式	结果关系
并	R和S必须是同类关系	与关系R (关系S) 的模式相同	$R \cup S = \{t \mid t \in R \vee t \in S\}$
差			$R - S = \{t \mid t \in R \wedge t \notin S\}$
交			$R \cap S = \{t \mid t \in R \wedge t \in S\}$

□ **同类关系**: 两个关系的关系模式同时满足以下两个要求

①具有相同的目n (具有相同数目的属性) ②相应的属性来自于同一个域

□ **并运算 和 交运算 满足 交换律 和 结合律**

➤ **交换律**: $R \cup S = S \cup R$, $R \cap S = S \cap R$

➤ **结合律**: $(R \cup S) \cup T = R \cup (S \cup T)$, $(R \cap S) \cap T = R \cap (S \cap T)$

□ **差运算 不满足 交换律 和 结合律**

‘连接’ 运算

□ ‘连接’ 运算又称 ‘ θ -连接’ 或 ‘theta-连接’，用于将关系 R 和关系 S 根据连接条件 F 合并为一个关系，可形式化表示为：

$$R \bowtie_F S$$

□ 连接运算是一种扩充关系运算，其功能可以用基本关系运算‘笛卡尔积+选择’来实现，可表示为：

$$R \bowtie_F S = \sigma_F(R \times S) \quad \text{或} \quad = (R \times S) \text{ where } F$$

□ 常见的连接运算

- θ -连接：按照给定的条件F实现两个关系之间的元组连接
- 等值连接：在连接条件F中，仅使用到等于比较 ‘=’ 这一种逻辑比较运算符
- 自然连接：两个关系之间的所有同名属性对之间的等值连接，且同名属性只保留一份在结果关系中
- 外连接：在结果集中同时含有自然连接结果元组和悬浮元组

□ 连接运算的含义

- 从两个关系的笛卡尔积中选取属性间满足一定条件的元组

$$R \bowtie_{A \theta B} S = \{ (t_r, t_s) \mid t_r \in R \wedge t_s \in S \wedge (t_r[A] \theta t_s[B]) \}$$

- A 和 B ：分别为 R 和 S 上度数相等且可比的属性组
- θ ：比较运算符

- 连接运算从 R 和 S 的广义笛卡尔积 $R \times S$ 中选取 R 关系在 A 属性组上的值与 S 关系在 B 属性组上的值满足比较关系 θ 的元组。

θ -连接 2

- 用 $Head(R)$ 来表示关系 R 的所有属性构成的集合（即：关系 R 的关系模式）
- 在关系 R 和 S 之间执行连接运算，结果关系为 T

$$T := R \bowtie_{A \theta B} S$$

➤ 结果关系 T 的关系模式

- $Head(T) = Head(R) \cup_{all} Head(S)$
- 不必消除它们之间的同名属性，但在结果关系中必须对同名属性进行换名

➤ 结果关系 T 的元组集合

- 从关系 R 和 S 中分别任取一个元组 t_r 和 t_s ，如果元组 t_r 和元组 t_s 的值能够使得连接条件 $A \theta B$ 成立，那么就由 t_r 和 t_s 合并构成一条结果元组，即：

$$R \bowtie_{A \theta B} S = \{ (t_r, t_s) \mid t_r \in R \wedge t_s \in S \wedge (t_r[A] \theta t_s[B]) \}$$

□ 关系连接运算：
$$R \bowtie_{A \theta B} S$$

□ 连接条件的构造方式

➤ 基本条件： $A \theta B$

- A 和 B ：来自于关系 R 和关系 S 的两个属性，也可以是常量
 - 一般情况下， A 和 B 分别来自于关系 R 和关系 S
 - 也允许 A 和 B 来自于同一个关系，或者一个是属性、一个是常量（相当于在同时执行‘关系连接 + 元组选择’），但不允许都是常量
- θ ：算术比较运算符
 - 根据所使用的比较运算符的不同，可以分别称之为：**等值连接**、**大于连接**，**小于连接**，.....

➤ 复杂条件：由若干个基本条件经逻辑运算符 $\wedge$ 和 $\vee$ 连接而成的复杂条件

□ 连接运算的推导公式

$$R \bowtie_F S = \sigma_F(R \times S) = (R \times S) \text{ where } F$$

□ 连接运算与笛卡儿积运算的关系

设 $T_1 = R \times S$, $T_2 = R \bowtie_F S$, 则

$$Head(T_2) = Head(T_1) \text{ 且 } T_2 \subseteq T_1$$

θ-连接 5：图2.7 (c)

例：关系 R 和关系 S

R	A	B	C
	a_1	b_1	5
	a_1	b_2	6
	a_2	b_3	8
	a_2	b_4	12

S	B	E
	b_1	3
	b_2	7
	b_3	10
	b_3	2
	b_5	2

连接 $R \bowtie_{C < E} S$ 的结果如下：

A	R.B	C	S.B	E
a_1	b_1	5	b_2	7
a_1	b_1	5	b_3	10
a_1	b_2	6	b_2	7
a_1	b_2	6	b_3	10
a_2	b_3	8	b_3	10

推导公式：

$$R \bowtie_{C < E} S = \pi_{R.A, R.B, R.C, S.B, S.E} (\sigma_{C < E} (R \times S))$$

□ 等值连接 (equi join)

- θ 为 “=” 的连接运算称为等值连接
- 从关系 R 与 S 的广义笛卡尔积中选取 A 、 B 属性值相等的那些元组，即等值连接为：

$$R \bowtie_{A=B} S = \{ (t_r, t_s) \mid t_r \in R \wedge t_s \in S \wedge t_r[A] = t_s[B] \}$$

等值连接 2: 图2.7 (d)

例: 关系R和关系S

R	A	B	C
	a_1	b_1	5
	a_1	b_2	6
	a_2	b_3	8
	a_2	b_4	12

S	B	E
	b_1	3
	b_2	7
	b_3	10
	b_3	2
	b_5	2

等值连接 $R \bowtie_{R.B = S.B} S$ 的结果如下:

A	R.B	C	S.B	E
a_1	b_1	5	b_1	3
a_1	b_2	6	b_2	7
a_2	b_3	8	b_3	10
a_2	b_3	8	b_3	2

推导公式:

$$R \bowtie_{R.B = S.B} S = \pi_{R.A, R.B, R.C, S.B, S.E} (\sigma_{R.B = S.B} (R \times S))$$

θ-连接 与 等值连接

关系R

A	B	C	D
1	2	3	4
3	2	1	8
7	3	2	1

关系S

E	F
1	8
7	9
5	2

【查询1】 $R \bowtie_{D > E} S$

A	B	C	D	E	F
1	2	3	4	1	8
3	2	1	8	1	8
3	2	1	8	7	9
3	2	1	8	5	2

【查询2】 $R \bowtie_{D = E} S$

A	B	C	D	E	F
7	3	1	2	1	8

□ 自然连接 (Natural join)

➤ 一种特殊的等值连接

- 两个关系中进行比较的分量必须是相同的属性组
- 在结果中把重复的属性列去掉

➤ 自然连接的含义

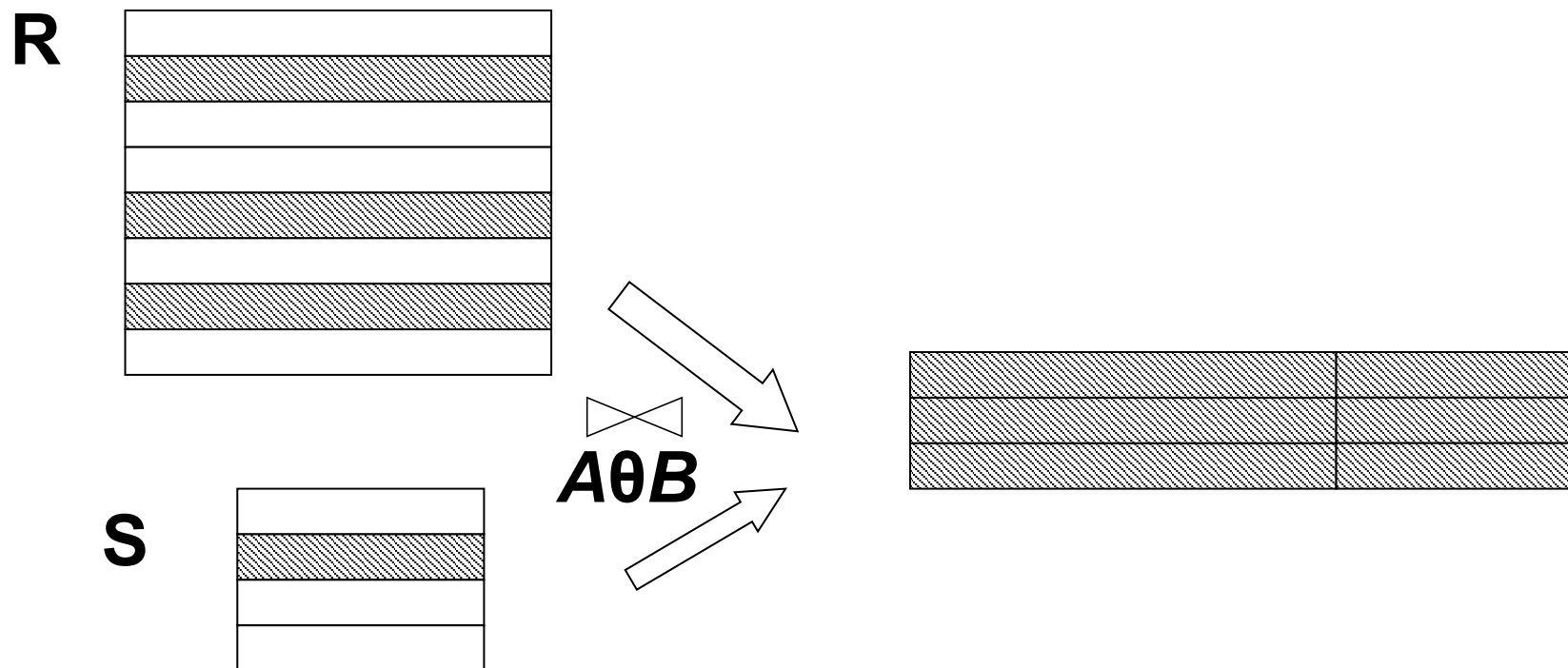
- R 和 S 含有相同的属性组 B

$$R \bowtie S = \{ \widehat{t_r t_s} [U - B] \mid t_r \in R \wedge t_s \in S \wedge t_r[B] = t_s[B] \}$$

- 其中： U 是两个关系所有属性的并集（包括相同的属性组 B ）

自然连接 2

□ 一般的连接操作是从行的角度进行运算。



自然连接还需要取消重复列，所以是同时从行和列的角度进行运算。

□ 自然连接运算： $R \bowtie S$

➤ 功能：根据两个关系中的同名属性（公共属性）进行等值连接

➤ 运算条件：关系R和关系S存在同名的公共属性，即：

$Head(R) \cap Head(S)$ 不是空集

➤ 运算结果

① 结果关系的关系模式： $Head(R) \cup Head(S)$

② 结果关系中的元组：

- 从关系R和关系S中分别任取一个元组 r 和 s ，如果 元组 r 和元组 s 在它们的同名属性上的取值都相等，那么可以由 r 和 s 合并构成结果关系中的一个元组
- 同名属性在结果关系中只保留一份

□ 自然连接运算的推导公式 $R \bowtie S$

➤ 设关系 R 和 S 的关系模式是：

- $Head(R) = \{A_1, A_2, \dots, A_n, C_1, C_2, \dots, C_k\}$
- $Head(S) = \{B_1, B_2, \dots, B_m, C_1, C_2, \dots, C_k\}$
- 它们之间的公共属性: $Head(R) \cap Head(S) = \{C_1, C_2, \dots, C_k\}$

➤ 结果关系的模式是：

$$Head(R \bowtie S) = \{A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m, C_1, C_2, \dots, C_k\}$$

➤ 结果关系的推导公式是：

$$R \bowtie S = \pi_{R.A_1, R.A_2, \dots, R.A_n, S.B_1, S.B_2, \dots, S.B_m, R.C_1, R.C_2, \dots, R.C_k} \left(\sigma_{R.C_1=S.C_1 \wedge R.C_2=S.C_2 \wedge \dots \wedge R.C_k=S.C_k} (R \times S) \right)$$

自然连接 5：图2.7

例：关系 R 和关系 S

R	A	B	C
	a_1	b_1	5
	a_1	b_2	6
	a_2	b_3	8
	a_2	b_4	12

S	B	E
	b_1	3
	b_2	7
	b_3	10
	b_3	2
	b_5	2

等值连接 $R \bowtie_{R.B = S.B} S$ 的结果如下：

A	R.B	C	S.B	E
a_1	b_1	5	b_1	3
a_1	b_2	6	b_2	7
a_2	b_3	8	b_3	10
a_2	b_3	8	b_3	2

自然连接 $R \bowtie S$ 的结果如下：

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2

推导公式： $R \bowtie S = \pi_{R.A, R.B, R.C, S.E} (\sigma_{R.B = S.B} (R \times S))$

自然连接 6

R

A	B	C
1	2	3
2	4	6
3	1	4

S

B	C	D
2	3	4
2	3	5
1	4	2
3	5	1

R ⋈ S

A	B	C	D
1	2	3	4
1	2	3	5
3	1	4	2

$$\begin{aligned}
 R \bowtie S &= \pi_{R.A, R.B, R.C, S.D} (\sigma_{R.B=S.B \wedge R.C=S.C} (R \times S)) \\
 &= ((R \times S) \text{ where } R.B = S.B \text{ and } R.C = S.C)[R.A, R.B, R.C, S.D]
 \end{aligned}$$

关系的‘自连接’ 1

- ❑ ‘自连接’ (self-join) 不是关系运算符，而是指一个关系自己和自己进行关系连接运算
- ❑ 在执行关系的自连接时，由于参与连接运算的是同一个关系R，它们具有相同的表名和表模式，无法直接使用常见的‘ θ -连接’或‘自然连接’来表示。
- ❑ 此时，需要使用赋值运算来为原表创建一个临时的中间表。在该中间表与原表之间：
 - 表模式相同但表名不同
 - 对应属性的属性名可以相同、也可以不同
- ❑ 然后，在原表和新创建的中间表之间执行关系连接运算

关系的‘自连接’ 2

❑ 例：在图2.4(a)所示的学生关系Student中，查询年龄不是最大的学生的学号。

(a) 学生关系 Student

学号 Sno	姓名 Sname	性别 Ssex	年龄 Sage	所在系 Sdept
201215121	李勇	男	20	CS
201215122	刘晨	女	19	CS
201215123	王敏	女	18	MA
201215125	张立	男	19	IS

中间关系 T

学号 Sno	姓名 Sname	性别 Ssex	年龄 Sage	所在系 Sdept
201215121	李勇	男	20	CS
201215122	刘晨	女	19	CS
201215123	王敏	女	18	MA
201215125	张立	男	19	IS

❑ 在这个查询中，需要比较两个学生元组的年龄大小，其中年龄小的就是本查询的目标元组。因此，需要使用两个学生关系并执行它们之间的 θ -连接（即学生关系上的自连接）。该查询可表示如下：

➤ 令 $T := Student$

➤ 使用 θ -连接，查询可表示为：

$$\pi_{Student.Sno} (Student \bowtie_{Student.Sage < T.Sage} T)$$

查询结果关系

学号 Sno
201215122
201215123
201215125

关系的‘自连接’ 3

- ❑ 例：在图2.4(a)所示的学生关系Student中，查询满足下述条件的学生的学号和姓名：年龄小于学号为201215121的学生。

(a) 学生关系 Student

学号 Sno	姓名 Sname	性别 Ssex	年龄 Sage	所在系 Sdept
201215121	李勇	男	20	CS
201215122	刘晨	女	19	CS
201215123	王敏	女	18	MA
201215125	张立	男	19	IS

- ❑ 在这个查询中，同样需要两次使用学生关系

- 查找学号为201215121的学生元组T
- 查找年龄小于T.Sage的学生元组

- ❑ 该查询可表示如下

- 令 $T := Student$
- 使用 θ -连接，查询可表示为：

$$\pi_{Student.Sno, Student.Sname} ((\sigma_{Sno='201215121'}(T)) \bowtie_{Student.Sage < T.Sage} Student)$$

- 也可直接使用基本运算表示如下：

$$\pi_{Student.Sno, Student.Sname} (\sigma_{T.Sno='201215121' \wedge Student.Sage < T.Sage} (T \times Student))$$

关系的‘自连接’ 4

- ❑ 例：在学生关系 **Student** 中，查询满足下述条件的学生的学号和姓名：与学号为 201215121 的学生就读于同一个院系。

(a) 学生关系 **Student**

学号 Sno	姓名 Sname	性别 Ssex	年龄 Sage	所在系 Sdept
201215121	李勇	男	20	CS
201215122	刘晨	女	19	CS
201215123	王敏	女	18	MA
201215125	张立	男	19	IS

- ❑ 同样，该查询也需使用到关系的自连接，可表示如下

➤ 令 $T := Student$

➤ 使用 θ -连接，查询可表示为：

$$\pi_{Student.Sno, Student.Sname} ((\sigma_{Sno='201215121'}(T)) \bowtie_{Student.Sdept = T.Sdept} Student)$$

- ❑ 在本例中，使用的是关系之间的等值连接，该查询也可使用‘自然连接’表示如下

➤ 令 $S := Student$, $T(no, Sdept) := \pi_{Sno, Sdept}(Student)$

➤ 使用自然连接，查询可表示为： $\pi_{S.Sno, S.Sname} (\sigma_{T.no='201215121'} (S \bowtie T))$

- ❑ 说明：请仔细体会恰当地使用赋值运算的好处。

外连接 1

□ 悬浮元组 (Dangling tuple)

- 两个关系 R 和 S 在做自然连接时，关系 R 中某些元组有可能在 S 中不存在公共属性上值相等的元组，从而造成 R 中这些元组在操作时被舍弃了，这些被舍弃的元组称为悬浮元组。

R		
A	B	C
a_1	b_1	5
a_1	b_2	6
a_2	b_3	8
a_2	b_4	12

S	
B	E
b_1	3
b_2	7
b_3	10
b_3	2
b_5	2

$R \bowtie S$			
A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2

□ 悬浮元组：关系 R 中的元组 $(a_2, b_4, 12)$ ，关系 S 中的元组 $(b_5, 2)$

□ 外连接 (Outer Join)

- 如果把悬浮元组也保存在结果关系中，而在其他属性上设置为空值 (Null)，这样的关系连接运算被称为‘外连接’。
- 左外连接 (LEFT OUTER JOIN 或 LEFT JOIN)
 - $R \text{ left join } S$: 只保留左边关系 R 中的悬浮元组
- 右外连接 (RIGHT OUTER JOIN 或 RIGHT JOIN)
 - $R \text{ right join } S$: 只保留右边关系 S 中的悬浮元组
- 外连接 (OUTER JOIN)
 - $R \text{ outer join } S$: 同时保留左右两边关系 R 和 S 中的悬浮元组

外连接 3

□ 图2.8: 表(a)是关系 R 和关系 S 的外连接的结果

表(a) R outer join S

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
a_2	b_4	12	NULL
NULL	b_5	NULL	2

外连接 4

□ 图2.8: 表(b)和表(c)分别是关系 R 和 S 的左外连接和右外连接的结果

表(b) R left outer join S

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
a_2	b_4	12	NULL

表(c) R right outer join S

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
NULL	b_5	NULL	2

□ 除运算的定义

- 教材中的定义
- 补充定义（扩展阅读）

□ 为什么要引入除运算

□ 如何正确使用除运算

除运算 1

□ 除运算 (Division)

- 设关系R除以关系S的结果为关系T，则T包含所有在R中但不在S中的属性及其值，且T的元组与S的元组的所有组合都在R中。

□ $T := R \div S$ 其中，R被称为‘被除数’，S被称为‘除数’，T被称为‘商’

- “T包含所有在R中但不在S中的属性及其值”

- $head(T) = head(R) - head(S)$
- if $t \in T$ then { 至少存在一个 $r \in R$ 且 $t = r[head(T)]$ }

- “T的元组与S的元组的所有组合都在R中”*

- for each $t \in T$ and $s \in S$ then $(t, s) \in R$

□ 除运算 (Division)

- 给定关系 $R(X, Y)$ 和 $S(Y, Z)$, 其中 X, Y, Z 为属性组。
- R 中的 Y 与 S 中的 Y 可以有不同的属性名, 但必须出自相同的域集 (在此情况下, 可以通过赋值运算创建中间关系来满足‘同名同义、异名异义’的约定, 再用中间关系参与后续的运算)
- R 与 S 的除运算得到一个新的关系 $P(X)$, P 是 R 中满足下列条件的元组在 X 属性列上的投影:

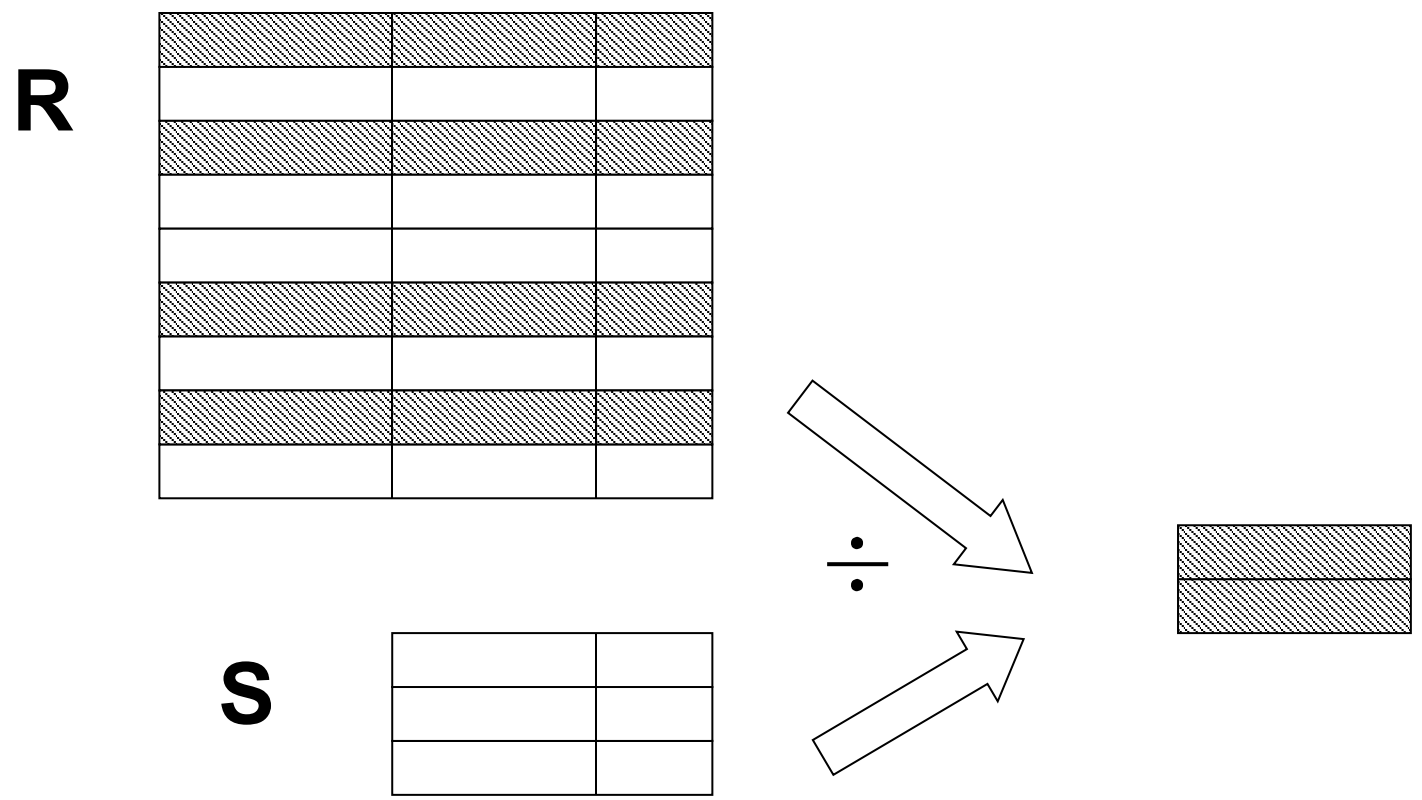
元组在 X 上分量值 x 的象集 Y_x 包含 S 在 Y 上投影的集合, 记作:

$$R \div S = \{ t_r[X] \mid t_r \in R \wedge \pi_Y(S) \subseteq Y_x \}$$

其中: Y_x 是 $x = t_r[X]$ 在 R 中的象集

除运算 3

❑除操作是同时从行和列角度进行运算



例2.9：设关系 R 、 S 分别为图2.9中的(a)和(b)， $R \div S$ 的结果为图(c)

图2.9 (a) 关系 R

A	B	C
a_1	b_1	c_2
a_2	b_3	c_7
a_3	b_4	c_6
a_1	b_2	c_3
a_4	b_6	c_6
a_2	b_2	c_3
a_1	b_2	c_1

图2.9 (b) 关系 S

B	C	D
b_1	c_2	d_1
b_2	c_1	d_1
b_2	c_3	d_2

图2.9 (c) $R \div S$

A
a_1

除运算 5：例2.9 $R \div S$ 的计算演示

□ 在关系 R 中， A 可以取四个值 $\{a_1, a_2, a_3, a_4\}$

➤ a_1 的象集为 $\{(b_1, c_2), (b_2, c_3), (b_2, c_1)\}$

➤ a_2 的象集为 $\{(b_3, c_7), (b_2, c_3)\}$

➤ a_3 的象集为 $\{(b_4, c_6)\}$

➤ a_4 的象集为 $\{(b_6, c_6)\}$

□ S 在 (B, C) 上的投影为：

$\{(b_1, c_2), (b_2, c_1), (b_2, c_3)\}$

□ 只有 a_1 的象集包含了 S 在 (B, C) 属性组上的投影

所以 $R \div S = \{a_1\}$

图2.9 (a) 关系 R

A	B	C
a_1	b_1	c_2
a_2	b_3	c_7
a_3	b_4	c_6
a_1	b_2	c_3
a_4	b_6	c_6
a_2	b_2	c_3
a_1	b_2	c_1

图2.9 (b) 关系 S

B	C	D
b_1	c_2	d_1
b_2	c_1	d_1
b_2	c_3	d_2

除运算 6：例2.9总结

例2.9：设关系 R 、 S 分别为图2.9中的(a)和(b)， $R \div S$ 的结果为图(c)

图2.9 (a) 关系 R

A	B	C
a_1	b_1	c_2
a_2	b_3	c_7
a_3	b_4	c_6
a_1	b_2	c_3
a_4	b_6	c_6
a_2	b_2	c_3
a_1	b_2	c_1

图2.9 (b) 关系 S

B	C	D
b_1	c_2	d_1
b_2	c_1	d_1
b_2	c_3	d_2

图2.9 (c) $R \div S$

A
a_1

□ 观察可以发现，仅出现在除数关系 S 中的属性 D 及其取值，与 $R \div S$ 的计算过程和计算结果（包括结果关系的关系模式和元组）都无关

- 在其他参考教材和资料中，一般要求“除数关系的关系模式必须是被除数关系模式的真子集”，即“当 $head(S) \subset head(R)$ 时， $R \div S$ 才是合法的”
- 在调用除运算前，可以在除数关系上调用投影运算来满足上述要求。如在例2.9中，其关系代数查询可表示为 $R \div \pi_{B,C}(S)$

除运算 7：例2.10

□ 以学生-课程数据库为例：

选修关系 SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

[例2.10] 查询至少选修1号课程和3号课程的学生号码

➤ 首先建立一个临时关系K：

Cno
1
3

➤ 然后求： $\pi_{Sno, Cno}(SC) \div K$

(计算过程见下一页)

除运算 8：例2.10的结果计算

□ 先执行查询 $\pi_{Sno, Cno}(SC)$, 结果如右表所示. 其中:

➤ 201215121 象集为 $\{1, 2, 3\}$

➤ 201215122 象集为 $\{2, 3\}$

➤ $K = \{1, 3\}$

Sno	Cno
201215121	1
201215121	2
201215121	3
201215122	2
201215122	3

□ 于是:

$$\pi_{Sno, Cno}(SC) \div K = \{201215121\}$$

除运算 9：补充定义 1

□ 除运算

- 用 $Head(R)$ 表示关系 R 的关系模式，用 $Head(S)$ 表示关系 S 的关系模式
- 当 $Head(S) \subset Head(R)$ 时，可以用关系 R 除以关系 S ： $R \div S$
- 其中：
 - R 被称为‘被除数关系’
 - S 被称为‘除数关系’
 - $R \div S$ 的结果关系 被称为‘商’
- 假设关系 R 和 S 的关系模式分别为：
$$Head(R) = \{ A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m \}$$
$$Head(S) = \{ B_1, B_2, \dots, B_m \}$$
- $R \div S$ 的结果关系的定义如下 (next slide)

除运算 10: 补充定义 2

□ 结果关系: $T = R \div S = \{ t_r[X] \mid t_r \in R \wedge \pi_Y(S) \subseteq Y_x \}$

➤ 结果关系的关系模式

$$Head(T) = Head(R) - Head(S) = \{ A_1, A_2, \dots, A_n \} = X$$

➤ 结果关系中的元组

- 满足条件的元组 t_r 在 X 属性列上的投影: 元组 t_r 在 X 上分量值 x 的象集 Y_x 包含 S 在 Y 上投影的集合 (根据前一页的假设, 在除数关系 S 中只有属性组 Y 即 $head(S)$, “ S 在 Y 上投影的集合” 就是关系 S)
- 即:

假设 x 是结果关系 T 中的一个元组 ($x \in T$), 则对于除数关系 S 中的每一个元组 y 都有 $(x, y) \in R$

➤ 由所有符合上述条件的元组 x 构成结果关系 T 的元组集合。

1. Assume a row x is in T , then:

for each row y in S

{

we can find a row z in R , and

{

$$z[A_1, A_2, \dots, A_n] = x[A_1, A_2, \dots, A_n]$$

$$z[B_1, B_2, \dots, B_m] = y[B_1, B_2, \dots, B_m]$$

}

}

2. T contains the largest possible set of rows x

扩展阅读：The Division Operation

Patrick O'Neil and Elizabeth O'Neil. Database: Principles, Programming and Performance.

□ The Division Operation

Consider two tables R and S, where the heading of S is a subset of the heading of R. Specifically, assume that

$$Head(R) = \{ A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m \}$$

$$Head(S) = \{ B_1, B_2, \dots, B_m \}$$

The table T is the result of the division $R \div S$ if

- ① $Head(T) = \{ A_1, A_2, \dots, A_n \}$, and
- ② T contains exactly those rows x such that for every row y in S, the row resulting from concatenating x and y can be found in table R.

$$(x, y) \in R$$

□ 除运算的定义

- 教材中的定义
- 补充定义（扩展阅读）

□ 为什么要引入除运算

□ 如何正确使用除运算

为什么要引入‘除’运算 1

□ 除运算： $R \div S$

□ ‘除’运算是关系代数中的一个扩充运算,为什么要引入‘除’运算？

➤ 为了方便表示某类查询：“选修过所有课程的学生”

□ 请比较下述查询的区别：

- ① 选修过1号课程的学生学号；
- ② 查询至少选修1号课程和3号课程的学生学号；
- ③ 选修过课程关系中的所有课程的学生学号；

选修关系 SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

为什么要引入‘除’运算 2

① 选修过1号课程的学生学号；

➤ 单表查询： $\pi_{Sno}(\sigma_{Cno='1'}(SC))$

② 查询至少选修1号课程和3号课程的学生学号；

➤ 方法1：单表查询 + 交运算

➤ 方法2：关系的连接

➤ 方法3：使用除运算（例2.10）

③ 选修过课程关系中的所有课程的学生学号；

➤ 可以使用除运算来方便查询③的表示（例2.13）

➤ 也可以不使用除运算，仅使用基本运算来表示查询③，但关系代数表达式比较复杂（请参考后面除运算的推导公式）

选修关系 SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

为什么要引入‘除’运算 3

② 查询至少选修1号课程和3号课程的学生学号；

➤ 查询表达式一：使用交运算

$$\pi_{Sno}(\sigma_{Cno='1'}(SC)) \cap \pi_{Sno}(\sigma_{Cno='3'}(SC))$$

➤ 查询表达式二：使用 θ -连接

令 $X := SC$ ，该查询可表示如下：

$$\pi_{SC.Sno}(SC \bowtie X)$$

$$\begin{aligned} & SC.Sno = X.sno \wedge \\ & SC.Cno = '1' \wedge \\ & X.Cno = '3' \end{aligned}$$

➤ 查询表达式三：使用自然连接

令 $X(Sno, xCno, xGrade) := SC$ ，该查询可表示如下：

$$\pi_{Sno}(\sigma_{Cno='1' \wedge xCno='3'}(SC \bowtie X))$$

选修关系 SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

为什么要引入‘除’运算 4

② 查询至少选修1号课程和3号课程的学生学号；

□ 查询表达式四 (例2.10)

➤ 首先建立一个临时关系 K ：

$$K := \pi_{Cno}(\sigma_{Cno=1 \vee Cno=3}(Course))$$

➤ 然后，该查询可表示为：

$$\pi_{Sno, Cno}(SC) \div K$$

➤ 将临时关系 K 带入后，得到如下的查询表达式：

$$\pi_{Sno, Cno}(SC) \div \pi_{Cno}(\sigma_{Cno=1 \vee Cno=3}(Course))$$

选修关系 SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

为什么要引入‘除’运算 5

③ 选修过课程关系中的所有课程的学生学号；

课程关系 Course

课程号 Cno	课程名 Cname	先行课 Cpno	学分 Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL语言	6	4

选修关系 SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

□ 使用除运算，该查询可表示如下：

$$\pi_{Sno, Cno}(SC) \div \pi_{Cno}(Course)$$

为什么要引入‘除’运算 6

① 选修过1号课程的学生学号;

➤ 单表查询: $\pi_{Sno}(\sigma_{Cno='1'}(SC))$

② 查询至少选修1号课程和3号课程的学生学号;

➤ 使用除运算:

$$\pi_{Sno, Cno}(SC) \div \pi_{Cno}(\sigma_{Cno=1 \vee Cno=3}(Course))$$

③ 选修过课程关系中的所有课程的学生学号;

➤ 使用除运算: $\pi_{Sno, Cno}(SC) \div \pi_{Cno}(Course)$

□ 思考:

➤ 为什么要引入除运算?

➤ 在调用除运算前, 为什么要先使用投影运算来得到除数和被除数关系?

如何正确使用除运算?

课程关系 Course

课程号 Cno	课程名 Cname	先行课 Cpno	学分 Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL语言	6	4

选修关系 SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

如何正确使用除运算 1

② 查询至少选修1号课程和3号课程的学生学号;

$$\pi_{Sno, Cno}(SC) \div \pi_{Cno}(\sigma_{Cno=1 \vee Cno=3}(Course))$$

□ 在调用除运算前，为什么要先使用投影运算来得到除数和被除数关系？

➤ 除数关系： $\pi_{Cno}(\sigma_{Cno=1 \vee Cno=3}(Course))$

- 确保 除数关系模式，符合除运算扩充定义中的要求

➤ 被除数： $\pi_{Sno, Cno}(SC)$

- 确保 结果元组，准确符合查询的语义
- 如果省略选课关系上的投影操作，则无法得到预期的查询结果 (next slide)

课程关系 Course

课程号 Cno	课程名 Cname	先行课 Cpno	学分 Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL语言	6	4

选修关系 SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

如何正确使用除运算 2

② 查询至少选修1号课程和3号课程的学生学号；

❑ 如果用例2.10中创建的临时关系K作为除数、用选课关系SC作为被除数，则查询例2.10可表示为： $\pi_{Sno}(SC \div K)$

❑ 其中，除运算 $SC \div K$ 的结果关系是：

➤ 结果关系的关系模式是： $\{Sno, Grade\}$

➤ 其中

- (201215121, 92) 的象集是 $\{1\}$
- (201215121, 85) 的象集是 $\{2\}$
- (201215121, 88) 的象集是 $\{3\}$
- (201215122, 90) 的象集是 $\{2\}$
- (201215122, 80) 的象集是 $\{3\}$

➤ 没有一个象集能够同时含有课程号1和3，因此，不存在符合除运算结果要求的元组。

临时关系 K

Cno
1
3

选修关系 SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

如何正确使用除运算 3

□ 设有如右所示的课程关系和选课关系

□ 观察下述三个关系代数表达式

① $SC \div Course$

② $SC \div \pi_{Cno}(Course)$

③ $\pi_{Sno, Cno}(SC) \div \pi_{Cno}(Course)$

【思考】

- 哪些是可以合法运算，哪些是非法的？
- 如果可以合法运算，那么
 - 结果关系的关系模式是什么？
 - 结果元组的语义是什么？

课程关系 Course

课程号 Cno	课程名 Cname	先行课 Cpno	学分 Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL语言	6	4

选修关系 SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

[例2.11] 查询选修了2号课程的学生们的学号。

$$\pi_{Sno}(\sigma_{Cno = '2'}(SC)) = \{201215121, 201215122\}$$

选修关系 SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

综合举例 2

[例2.12] 查询至少选修了一门其直接先行课为5号课程的学生姓名

学生关系 Student

学号 Sno	姓名 Sname	性别 Ssex	年龄 Sage	所在系 Sdept
201215121	李勇	男	20	CS
201215122	刘晨	女	19	CS
201215123	王敏	女	18	MA
201215125	张立	男	19	IS

课程关系 Course

课程号 Cno	课程名 Cname	先行课 Cpno	学分 Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL语言	6	4

选修关系 SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

□ 本题查询可以分三步来完成：

- ① 查询先行课为5号课程的课程： $T_1 := \sigma_{Cpno = '5'}(Course)$
- ② 查找 T_1 中所有课程的选课记录： $T_2 := T_1 \bowtie SC$
- ③ 查找 T_2 中选课元组上学生的姓名： $\pi_{Sname}(T_2 \bowtie Student)$

综合举例 3

[例2.12] 查询至少选修了一门其直接先行课为5号课程的学生姓名

学生关系 Student

学号 Sno	姓名 Sname	性别 Ssex	年龄 Sage	所在系 Sdept
201215121	李勇	男	20	CS
201215122	刘晨	女	19	CS
201215123	王敏	女	18	MA
201215125	张立	男	19	IS

课程关系 Course

课程号 Cno	课程名 Cname	先行课 Cpno	学分 Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL语言	6	4

选修关系 SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

□ 合并后可得到本题的查询表示如下：

$$\pi_{Sname} ((\sigma_{Cpno = '5'} (Course) \bowtie SC) \bowtie Student)$$

□ 也可表示为： $\pi_{Sname} (\sigma_{Cpno = '5'} (Course \bowtie SC \bowtie Student))$

或 $\pi_{Sname} (\pi_{Sno} (\sigma_{Cpno = '5'} (Course) \bowtie SC) \bowtie \pi_{Sno, Sname} (Student))$

综合举例 4

[例2.13] 查询选修了全部课程的学生号码和姓名。

$$(\pi_{Sno, Cno}(SC) \div \pi_{Cno}(Course)) \bowtie \pi_{Sno, Sname}(Student)$$

学生关系 Student

学号 Sno	姓名 Sname	性别 Ssex	年龄 Sage	所在系 Sdept
201215121	李勇	男	20	CS
201215122	刘晨	女	19	CS
201215123	王敏	女	18	MA
201215125	张立	男	19	IS

课程关系 Course

课程号 Cno	课程名 Cname	先行课 Cpno	学分 Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL语言	6	4

选修关系 SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

□ 也可以表示如下：

$$\pi_{Sno, Sname, Cno}(SC \bowtie Student) \div \pi_{Cno}(Course)$$

扩充运算 (小结)

□ 交运算: $R \cap S = \{ t \mid t \in R \wedge t \in S \}$

□ θ -连接: $R \bowtie_{A \theta B} S = \{ (t_r, t_s) \mid t_r \in R \wedge t_s \in S \wedge (t_r[A] \theta t_s[B]) \}$

□ 自然连接:

$$R \bowtie S = \{ (t_r[A], t_r[B], t_s[C]) \mid t_r \in R \wedge t_s \in S \wedge (t_r[B] = t_s[B]) \}$$

➤ 其中: 关系R的属性被划分为A和B两个属性组, 关系S的属性被划分为B和C两个属性组, 属性组B为两者之间的同名公共属性

□ 除运算: 给定关系 $R(X, Y)$ 和 $S(Y)$, 其中 X, Y 为属性组。

$$R \div S = \{ t_r[X] \mid t_r \in R \wedge \pi_Y(S) \subseteq Y_x \}$$

其中: Y_x 是 $x = t_r[X]$ 在 R 中的象集

一. 关系代数的扩充运算

二. 除运算的推导

三. 扩充运算的应用

四. 关系代数小结

二、除运算的推导公式 1

- '除' 运算是关系代数中的一个扩充运算，它的推导公式如下
- 设关系 R 的属性集为 $\{A_1, \dots, A_n, B_1, \dots, B_m\}$ ，关系 S 的属性集为 $\{B_1, \dots, B_m\}$ ，则：

$$R \div S = \pi_{A_1, \dots, A_n}(R) - \pi_{A_1, \dots, A_n}((\pi_{A_1, \dots, A_n}(R) \times S) - R)$$

其推导过程如下：

二、除运算的推导公式 2

$$\textcircled{1} \quad T_{\max} := \pi_{A_1, \dots, A_n}(R)$$

// $T_{\max}$ 是最大可能的结果元组集合

$$\textcircled{2} \quad R_{\max} := T_{\max} \times S$$

// $R_{\max}$ 与关系 R 是同类关系

$$\textcircled{3} \quad T_1 := R_{\max} - R$$

$$\textcircled{4} \quad T_2 := \pi_{A_1, \dots, A_n}(T_1)$$

// T_2 是关系 $T_{\max}$ 中不满足除运算的结果要求的那些元组，即：对于关系 T_2 中的任一个元组 q ，至少能在关系 S 中找到一个元组 s ，使得由元组 q 和 s 所构成的元组 (q, s) 不在关系 R 中出现。

$$\textcircled{5} \quad R \div S := T_{\max} - T_2$$

除运算的推导过程 (1) $SC \div C$

SC

sno	cno
s_1	c_1
s_1	c_2
s_2	c_1
s_2	c_2
s_2	c_3
s_3	c_2

C

cno
c_1
c_2
c_3

T_{max}

sno
s_1
s_2
s_3

SC_{max}

sno	cno
s_1	c_1
s_1	c_2
s_1	c_3
s_2	c_1
s_2	c_2
s_2	c_3
s_3	c_1
s_3	c_2
s_3	c_3

$\pi_{sno}(SC_{max} - SC)$

sno
s_1
s_3

结果关系T

sno
s_2

没有出现在SC中的元组，其中的sno不符合‘除’运算对结果元组的要求。

- ❑ 如果保留被除数关系中的课程成绩属性G（如下图所示的关系R），那么除法的结果语义就会产生改变！（计算过程如下）

R		
sno	cno	G
s_1	c_1	80
s_1	c_2	85
s_2	c_1	90
s_2	c_2	70
s_2	c_3	85
s_3	c_2	85

C
cno
c_1
c_2

$$T_{max} = \pi_{sno, G}(R)$$

sno	G
s_1	80
s_1	85
s_2	90
s_2	70
s_2	85
s_3	85

$$R$$

sno	cno	G
s_1	c_1	80
s_1	c_2	85
s_2	c_1	90
s_2	c_2	70
s_2	c_3	85
s_3	c_2	85

$$C$$

cno
c_1
c_2
c_3

$$T_{max} = \pi_{sno,G}(R)$$

sno	G
s_1	80
s_1	85
s_2	90
s_2	70
s_2	85
s_3	85

$$R_{max} = T_{max} \times C$$

sno	cno	G
s_1	c_1	80
s_1	c_2	80
s_1	c_3	80
s_1	c_1	85
s_1	c_2	85
s_1	c_3	85
s_2	c_1	90
s_2	c_2	90
s_2	c_3	90
s_2	c_1	70
s_2	c_2	70
s_2	c_3	70
s_2	c_1	85
s_2	c_2	85
s_2	c_3	85
s_3	c_1	85
s_3	c_2	85
s_3	c_3	85

- 从 R_{max} 中，我们无法判断哪些学生是选修过课程关系 C 中的所有三门课！
- 直接用关系 R 作为被除数，无法准确表示“选修过关系 C 中的所有课程”的查询语义，正确的表示方法是：

$$\pi_{sno,cno}(R) \div C$$

三、扩充运算的应用

❑ 【数据库一】学生选课数据库的关系模式

- 学生关系：S (学号 sno, 姓名 sn, 就读院系 sd, 年龄 sa)
- 课程关系：C (课程号 cno, 课程名 cn, 先修课程号 pno)
- 选课关系：SC (学号 sno, 课程号 cno, 成绩 g)

请用关系代数来表示下述数据操纵请求：

- 1.1 查询计算机学院年龄小于20岁的学生的学号、姓名和年龄
- 1.2 查询计算机学院所有学生的课程修读情况(学号,姓名,课程号,课程名,成绩)
- 1.3 查询有不及格课程（成绩小于60分）的学生，结果返回这些学生的学号和姓名
- 1.4 查询课程成绩都优秀（成绩大于或等于90分）的学生，结果返回这些学生的姓名和就读院系
- 1.5 查询学号为201400101的学生修读过的课程的课程号、课程名、成绩
- 1.6 查询至少修读过‘学生201400101修读过的一门课’的学生的姓名
- 1.7 查询修读过‘学生201400101修读过的所有课程’的学生的姓名
- 1.8 在关系C中增添一门新课程 (c30013, MachineLearning, c10002)
- 1.9 请在学生关系S和选课关系SC中删除与学生201900123有关的所有元组
- 1.10 将关系S中学生202000101的年龄改为22岁

三、扩充运算的应用

学生 S (sno, sn, sd, sa)

课程 C (cno, cn, pno)

选课 SC (sno, cno, g)

1.1 查询计算机学院年龄小于20岁的学生的学号、姓名和年龄

$\pi_{sno, sn, sa} (\sigma_{sd='计算机学院' \wedge sa < 20} (S))$

1.2 查询计算机学院所有学生的课程修读情况(学号,姓名,课程号,课程名,成绩)

$\pi_{sno, sn, cno, cn, g} (\sigma_{sd='计算机学院'} (S \bowtie SC \bowtie C))$

三、扩充运算的应用

学生 S (sno, sn, sd, sa)

课程 C (cno, cn, pno)

选课 SC (sno, cno, g)

1.3 查询有不及格课程（成绩小于60分）的学生，结果返回这些学生的学号和姓名

$$\pi_{sno, sn} (\sigma_{g < 60} (S \bowtie SC))$$

1.4 查询课程成绩都优秀（成绩大于或等于90分）的学生，结果返回这些学生的姓名和就读院系

$$\pi_{sn, sd} ((\pi_{sno} (SC) - \pi_{sno} (\sigma_{g < 90} (SC))) \bowtie S)$$

➤ 错误1: $\pi_{sn, sd} (\sigma_{g \geq 90} (S \bowtie SC))$

➤ 错误2: $\pi_{sn, sd} (S \bowtie SC) - \pi_{sn, sd} (\sigma_{g < 90} (S \bowtie SC))$

➤ 错误3: $\pi_{sn, sd} ((\pi_{sno} (S) - \pi_{sno} (\sigma_{g < 90} (SC))) \bowtie S)$

三、扩充运算的应用

学生 S (sno, sn, sd, sa)

课程 C (cno, cn, pno)

选课 SC (sno, cno, g)

1.5 查询学号为201400101的学生修读过的课程的课程号、课程名、成绩

$\pi_{C.cno, C.cn, SC.g} (\sigma_{sno='201400101'} (SC \bowtie C))$

1.6 查询至少修读过‘学生201400101修读过的一门课程’的学生的姓名

【分析】

① 结果元组（学生）需要满足的条件是什么？

修读过‘学生201400101修读过的某一门课程’

② 学生201400101修读过哪些课程？

③ 如何从学生201400101修读过的课程查出满足要求的学生元组？

【例 1.6】

1.6 查询至少修读过‘学生201400101修读过的一门课’的学生的姓名

① 查询学生201400101修读过的课程的课程号

$$T_1 := \pi_{cno} (\sigma_{sno='201400101'} (SC))$$

② 查询选修过 T_1 中课程的学生的学号

$$T_2 := \pi_{sno} (SC \bowtie T_1)$$

③ 查询 T_2 中学生的姓名（即本题的查询结果）

$$\pi_{sn} (S \bowtie T_2)$$

➤ 将上述表达式依次代入后得到最终的查询表达式如下：

$$\pi_{sn} (S \bowtie (\pi_{sno} (SC \bowtie \pi_{cno} (\sigma_{sno='201400101'} (SC)))))$$

三、扩充运算的应用

1.7 查询修读过‘学生201400101修读过的所有课程’的学生的姓名

【分析】

- ① 结果元组需要满足的条件？
修读过‘学生201400101修读过的所有课程’
- ② 学生201400101修读过哪些课程？
- ③ 如何根据一组课程（学生201400101修读过的所有课程）查出修读过其中的所有课程的学生姓名？

【答】

$$\pi_{sn} (S \bowtie (\pi_{sno, cno} (SC) \div \pi_{cno} (\sigma_{sno='201400101'} (SC)))))$$

【例 1.7】思考

1.7 $\pi_{sn}(S \bowtie (\pi_{sno, cno}(SC) \div \pi_{cno}(\sigma_{sno='201400101'}(SC))))$

本题的以下几种表示方法都是错误的，为什么？

① $\pi_{sn}(S \bowtie \pi_{sno}(SC \bowtie \pi_{cno}(\sigma_{sno='201400101'}(SC))))$

② $\pi_{sn}(S \bowtie (SC \div \pi_{cno}(\sigma_{sno='201400101'}(SC))))$

③ $\pi_{sn}(S \bowtie (\pi_{sno, cno}(SC) \div \sigma_{sno='201400101'}(SC)))$

④ $\pi_{sn}(S \bowtie (SC \div \sigma_{sno='201400101'}(SC)))$

⑤ $\pi_{sn}((S \bowtie SC) \div \pi_{cno}(\sigma_{sno='201400101'}(SC)))$

⑥ $\pi_{sn, cno}(S \bowtie SC) \div \pi_{cno}(\sigma_{sno='201400101'}(SC))$

【思考】不使用‘除’运算，该怎么表示该查询？

参考答案

例1.5、1.6、1.7 的对比总结

□ 请仔细分析、理解查询1.5、1.6 和 1.7 的语义及其相互之间的区别。

1.5 查询学号为201400101的学生修读过的课程的课程号、课程名、成绩

$$\pi_{C.cno, C.cn, SC.g} (\sigma_{sno='201400101'} (SC \bowtie C))$$

1.6 查询至少修读过 ‘学生201400101修读过的一门课’ 的学生的姓名

$$\pi_{sn} (S \bowtie (\pi_{sno} (SC \bowtie \pi_{cno} (\sigma_{sno='201400101'} (SC)))))$$

1.7 查询修读过 ‘学生201400101修读过的所有课程’ 的学生的姓名

$$\pi_{sn} (S \bowtie (\pi_{sno, cno} (SC) \div \pi_{cno} (\sigma_{sno='201400101'} (SC))))$$

三、扩充运算的应用

学生 $S(sno, sn, sd, sa)$ 课程 $C(cno, cn, pno)$ 选课 $SC(sno, cno, g)$

可以用关系代数表达式来表示关系上的元组插入、删除、修改操作。

1.8 在关系C中增添一门新课程 (c30013, MachineLearning, c10002)

$$C \cup ('c30013', 'MachineLearning', 'c10002')$$

1.9 请在学生关系S和选课关系SC中删除与学生201900123有关的所有元组

$$SC - \sigma_{sno='201900123'}(SC)$$

$$S - \sigma_{sno='201900123'}(S)$$

三、扩充运算的应用

1.10 将关系S中学生202000101的年龄改为22岁

【分析】

- ① 查询学号为202000101的学生: $T_1 := \sigma_{sno='202000101'}(S)$
- ② 修改后的学生202000101元组构成关系 W
- ③ 将修改后的新元组插入到关系S中, 插入后的结果关系为: $(S - T_1) \cup W$

合并代入后如下:

$$(S - \sigma_{sno='202000101'}(S)) \cup W$$

三、扩充运算的应用

□ 【数据库二】 关系模式

➤ 顾客 (编号, 姓名, 顾客所在城市, 折扣)

C(cid, cn, city, dis)

➤ 供应商 (编号, 名称, 供应商所在城市, 佣金比例)

A(aid, an, city, per)

➤ 商品 (编号, 名称, 商品库存所在城市, 库存量, 单价)

P(pid, pn, city, qua, pri)

➤ 订单 (订单编号, 订购日期, 顾客编号, 供应商编号,
商品编号, 订购数量, 订单金额)

O(ordno, orddate, cid, aid, pid, qty, dol)

三、扩充运算的应用

请用关系代数来表示下述数据操纵请求：

- 2.1 检索销售过‘c004’号顾客购买过的所有商品的供应商的编号
- 2.2 检索购买过 'p01' 和 'p07' 号两种商品的顾客的编号
- 2.3 检索购买过 ‘p01’ 或 ‘p07’ 号商品的顾客的编号
- 2.4 检索购买过p02号商品的顾客所在的城市及销售过p02号商品的供应商所在城市
- 2.5 检索至少为购买过p01号商品的某一位顾客订购过商品的供应商的编号
- 2.6 检索为购买过p01号商品的所有顾客订购过商品的供应商的编号
- 2.7 检索具有最高折扣(discont)的顾客的编号
- 2.8 检索至今为止没有通过 ‘a03’ 号供应商来购买过商品的顾客的编号
- 2.9 检索仅通过 ‘a03’ 号供应商来购买过商品的顾客的编号
- 2.10 检索至少购买过一种价格为\$0.50的商品的顾客的名称
- 2.11 检索购买过所有价格为\$0.50的商品的顾客的姓名

三、扩充运算的应用

顾客 C(cid, cn, city, dis)

供应商 A(aid, an, city, per)

商品 P(pid, pn, city, qua, pri)

订单 O(ordno, orddate, cid, aid, pid, qty, dol)

2.1 检索销售过‘c004’号顾客购买过的所有商品的供应商的编号

$$\pi_{aid, pid}(O) \div \pi_{pid}(\sigma_{cid='c004'}(O))$$

【思考】不使用‘除’运算，仅使用基本运算如何表示该查询？

$$\pi_{aid}(O) - \pi_{aid}((\pi_{aid}(A) \times \pi_{pid}(\sigma_{cid='c004'}(O)))) - \pi_{aid, pid}(O)$$

2.2 检索购买过 'p01' 和 'p07' 号两种商品的顾客的编号

$$\pi_{cid}(\sigma_{pid='p01'}(O)) \cap \pi_{cid}(\sigma_{pid='p07'}(O))$$

□ 请检查下述关系代数表达式，是否也能满足本题的查询要求？

[1] $\pi_{cid}(\sigma_{pid='p01' \text{ and } pid='p07'}(O))$

✗

[2] $\pi_{cid}(\sigma_{pid='p01' \text{ or } pid='p07'}(O))$

✗

[3] $\pi_{cid}(O \bowtie \pi_{pid}(\sigma_{pid='p01' \text{ or } pid='p07'}(P)))$

✗

[4] $\pi_{cid,pid}(O) \div \pi_{pid}(\sigma_{pid='p01' \text{ or } pid='p07'}(P))$

✓

[5] 令 $S := O$ $\pi_{O.cid}((\sigma_{pid='p01'}(O)) \bowtie_{S.cid = O.cid} (\sigma_{pid='p07'}(S)))$

✓

[6] $\pi_{cid}(\sigma_{pid='p01'}(O) \cap \sigma_{pid='p07'}(O))$

✗

2.3 检索购买过 'p01' 或 'p07' 号商品的顾客的编号

$$\pi_{cid} (\sigma_{pid = 'p01'} (O)) \cup \pi_{cid} (\sigma_{pid = 'p07'} (O))$$

□ 请检查下述关系代数表达式，是否也能满足本题的查询要求？

[1] $\pi_{cid} (\sigma_{pid = 'p01' \text{ and } pid = 'p07'} (O))$

✗

[2] $\pi_{cid} (\sigma_{pid = 'p01' \text{ or } pid = 'p07'} (O))$

✓

[3] $\pi_{cid} (O \bowtie \pi_{pid} (\sigma_{pid = 'p01' \text{ or } pid = 'p07'} (P)))$

✓

[4] $\pi_{cid,pid} (O) \div \pi_{pid} (\sigma_{pid = 'p01' \text{ or } pid = 'p07'} (P))$

✗

[5] $\pi_{cid} (\sigma_{pid = 'p01'} (O) \cup \sigma_{pid = 'p07'} (O))$

✓

□ 查询满足下述条件的顾客的编号：

参考答案

- ① 购买过 'p01' 号商品
- ② 没有购买过 'p01' 号商品
- ③ 只购买过 'p01' 号商品

- ④ 既购买过 'p01' 也购买过 'p02' 号商品
- ⑤ 既没有购买过 'p01'，也没有购买过 'p02' 号商品
- ⑥ 只购买过 'p01' 和 'p02' 号两种商品

- ⑦ 没有购买过商品
- ⑧ 只购买过一次商品（只有一份订单）
- ⑨ 只购买过同一种商品

三、扩充运算的应用

顾客 C(cid, cn, city, dis)

供应商 A(aid, an, city, per)

商品 P(pid, pn, city, qua, pri)

订单 O(ordno, orddate, cid, aid, pid, qty, dol)

2.4 检索购买过p02号商品的顾客所在的城市及销售过p02号商品的供应商所在城市

$$\pi_{\text{city}} (\sigma_{\text{pid}='p02'} (O) \bowtie C) \cup \pi_{\text{city}} (\sigma_{\text{pid}='p02'} (O) \bowtie A)$$

2.5 检索至少为购买过p01号商品的某一位顾客订购过商品的供应商的编号

$$\pi_{\text{aid}} ((\pi_{\text{cid}} (\sigma_{\text{pid}='p01'} (O))) \bowtie O)$$

2.6 检索为购买过p01号商品的所有顾客订购过商品的供应商的编号

$$\pi_{\text{cid,aid}} (O) \div \pi_{\text{cid}} (\sigma_{\text{pid}='p01'} (O))$$

三、扩充运算的应用

顾客 $C(cid, cn, city, dis)$

供应商 $A(aid, an, city, per)$

商品 $P(pid, pn, city, qua, pri)$

订单 $O(ordno, orddate, cid, aid, pid, qty, dol)$

2.7 检索具有最高折扣(discnt)的顾客的编号

(解1) $\pi_{cid}(C) - \pi_{C.cid}(\sigma_{C.discnt < S.discnt}(C \times S))$

(解2) 使用除运算：令 $M := C, N := C$

① 查找折扣存在大小关系的客户对：

$T(bigid, cid) := \pi_{M.cid, N.cid}(\sigma_{M.discnt \geq N.discnt}(M \times N))$

② 使用除运算查询得到折扣最高的顾客的编号： $T \div \pi_{cid}(C)$

三、扩充运算的应用

顾客 $C(cid, cn, city, dis)$

供应商 $A(aid, an, city, per)$

商品 $P(pid, pn, city, qua, pri)$

订单 $O(ordno, orddate, cid, aid, pid, qty, dol)$

2.8 检索至今为止没有通过‘a03’号供应商来购买过商品的顾客的编号

$$\pi_{cid}(C) - \pi_{cid}(\sigma_{aid='a03'}(O))$$

➤ 2.8不能错误表示为: $\pi_{cid}(O) - \pi_{cid}(\sigma_{aid='a03'}(O))$

2.9 检索仅通过‘a03’号供应商来购买过商品的顾客的编号

$$\pi_{cid}(O) - \pi_{cid}(\sigma_{aid \neq 'a03'}(O))$$

➤ 2.9不能错误表示为: $\pi_{cid}(C) - \pi_{cid}(\sigma_{aid \neq 'a03'}(O))$

三、扩充运算的应用

顾客 $C(cid, cn, city, dis)$

供应商 $A(aid, an, city, per)$

商品 $P(pid, pn, city, qua, pri)$

订单 $O(ordno, orddate, cid, aid, pid, qty, dol)$

2.10 检索至少购买过一种价格为\$0.50的商品的顾客的名称

$$\pi_{cn} \left(\left(\left(\pi_{pid} \left(\sigma_{pri=0.5} (P) \right) \right) \bowtie O \right) \bowtie C \right)$$

2.11 检索购买过所有价格为\$0.50的商品的顾客的姓名

① 先查出满足条件的顾客的编号: $T_1 := \pi_{cid, pid} (O) \div \pi_{pid} (\sigma_{pri=0.5} (P))$

② 再查出这些满足条件的顾客的姓名: $T_2 := \pi_{cn} (T_1 \bowtie C)$

四、关系代数总结

□ 关系的表示

- 元组的集合

□ 关系操纵的表示

- 集合上的运算

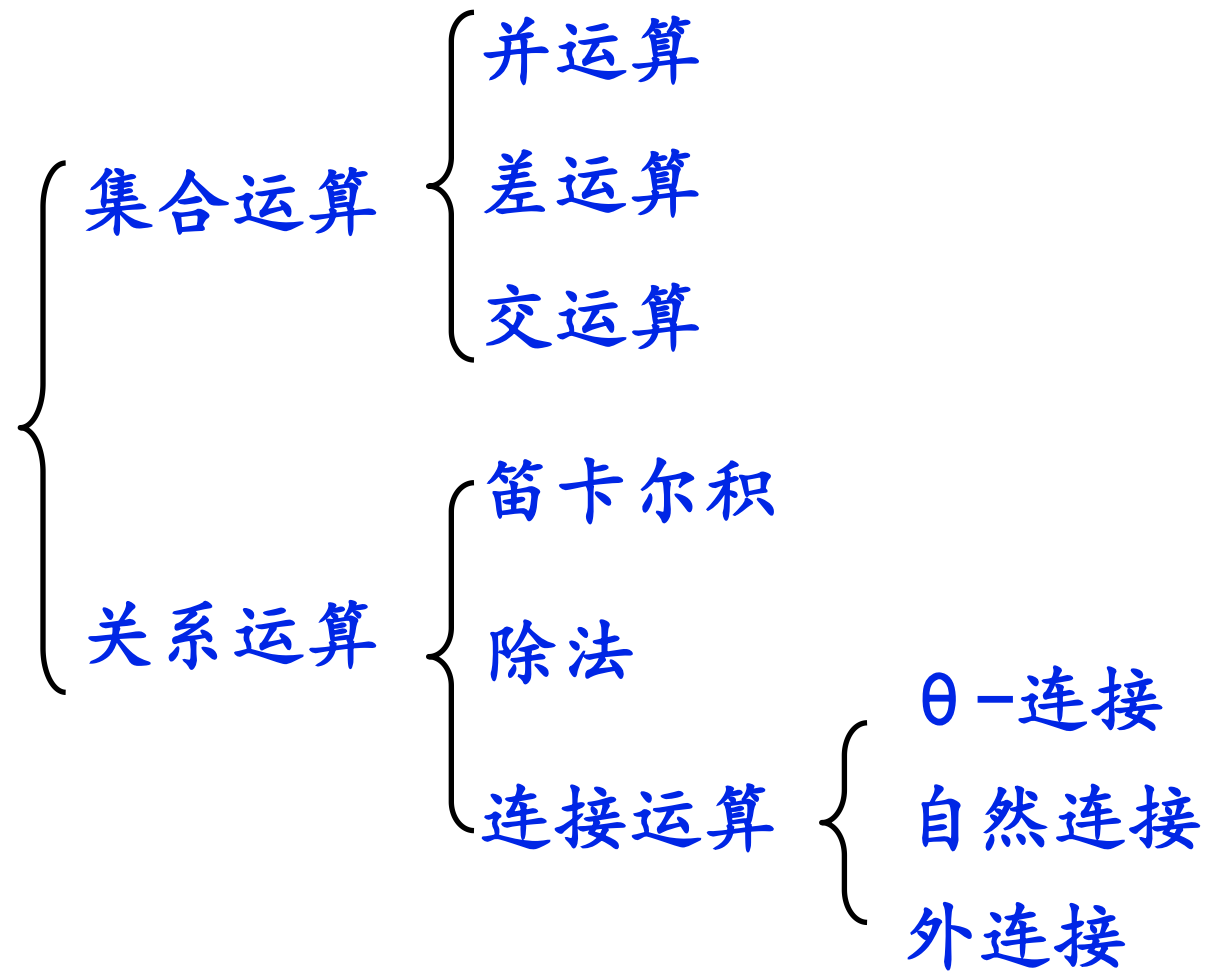
□ 五种基本运算

- 并 $\cup$ ，差 $-$
- 投影 π ，选择 σ ，笛卡儿积 $\times$

□ 五种扩充运算

- 交 $\cap$
- 除 $\div$
- θ -连接，自然连接 $\bowtie$ ，外连接，外连接

关系的合并运算



□ When & How

- 投影 / 选择
- 并 / 交 / 差
- 笛卡尔积 / θ -连接 / 自然连接 / 除法

□ Difference

- 笛卡尔积 vs. 自然连接
- 自然连接 vs. 除法
- 差运算 vs. ‘不等’比较 (not equal)

特殊运算（差）的使用方法

□ When ?

- 当查询条件带有‘否定’语义，或者具有明显的‘排它性’的时候，通常需要使用两个子查询之间的‘差’运算

□ How ?

- ‘差’运算的运算对象（关系）中，通常需要包含其码

“笛卡尔积/ θ -连接/自然连接”的使用方法

□ 都是关系的合并运算

- 笛卡尔积是基本运算， θ -连接和自然连接则是扩充运算
- 请注意三者的结果关系的关系模式之间的区别

□ 笛卡尔积

- 是实现跨不同关系表进行数据访问的基础
- 在笛卡尔积的结果关系中，存在着很多无意义的结果元组，通常需要通过后续的选择运算过滤掉

□ θ -连接

- 相邻的‘笛卡尔积+选择运算’可以合并为一个 θ -连接

□ 自然连接

- 如果连接条件是基于“两张表中的所有同名属性的相等比较”，可以将 θ -连接进一步简写为自然连接

“笛卡尔积/ θ -连接/自然连接” (cont.)

❑ 一般方法：笛卡尔积+选择 or θ -连接

- 不存在同名属性，或者连接条件不是基于同名属性的相等比较
- 在结果关系中可能存在同名属性，需要加以区别

❑ 常用方法：自然连接

- 连接条件是隐含的（所有同名属性的相等比较）
- 如果在两个关系之间存在多对‘同名属性’，而本次查询又不需要‘所有’的同名属性都相等，此时有两种选择：
 - ① 采用前述的一般方法来实现关系的合并
 - ② 先对其中的一个关系执行投影运算，过滤掉其中不需要相等的那些同名属性，然后再使用自然连接运算

❑ 难点：关系的自连接

- 使用赋值运算定义‘同质不同名’的两个中间关系(元组集合相同，但关系名不同)，当然也可以对中间关系中的属性进行重命名
- 然后再使用前述的一般方法实现两个中间关系的合并

特殊运算（除）的使用方法

□ ‘除’ 运算与 ‘连接’ 运算的区别

- 我们将查询的结果关系称为‘目标对象’，用于定义查询条件的关系称为‘条件对象’
- 在决定某个元组t是否属于结果关系时，
 - 如果只需要从条件对象中找到一个元组c并使得查询条件成立，那么就直接使用‘连接’运算（包括笛卡尔积、 θ -连接和自然连接）
 - 如果需要条件对象集中的所有元组都能使得查询条件成立，那么就使用‘除’运算

□ ‘除’ 运算表达式的表示方法

- 被除数关系中必须包含目标对象和条件对象的码
- 除数关系中只含条件对象的码
- 被除数和除数关系中不能含其它‘不必要’的多余属性

1.7 查询修读过‘学生201400101修读过的所有课程’的学生的姓名

返回

$$\pi_{sn}(S \bowtie (\pi_{sno, cno}(SC) \div \pi_{cno}(\sigma_{sno='201400101'}(SC))))$$

【思考】不使用‘除’运算，该查询可分步表示如下：

① 学生201400101修读过的所有课程：

$$T_1 := \pi_{cno}(\sigma_{sno='201400101'}(SC))$$

② 查询符合下述条件的学生的学号：存在 T_1 中的某些课程，该同学没有修读过

$$T_2 := \pi_{sno}((\pi_{sno}(S) \times T_1) - \pi_{sno, cno}(SC))$$

③ 查询修读过‘学生201400101修读过的所有课程’的学生的学号： $\pi_{sno}(SC) - T_2$

④ 查询上述学生的姓名： $\pi_{sn}(S \bowtie (\pi_{sno}(SC) - T_2))$

代入后可得到如下的查询表达式：

$$\pi_{sn}(S \bowtie (\pi_{sno}(SC) - \pi_{sno}((\pi_{sno}(S) \times \pi_{cno}(\sigma_{sno='201400101'}(SC))) - \pi_{sno, cno}(SC))))$$

□ 查询满足下述条件的顾客的编号：

① 购买过 'p01' 号商品： $\pi_{cid}(\sigma_{pid='p01'}(O))$

② 没有购买过 'p01' 号商品： $\pi_{cid}(C) - \pi_{cid}(\sigma_{pid='p01'}(O))$

③ 只购买过 'p01' 号商品： $\pi_{cid}(O) - \pi_{cid}(\sigma_{pid \neq 'p01'}(O))$

④ 既购买过 'p01' 也购买过 'p02' 号商品：

$$\pi_{cid}(\sigma_{pid='p01'}(O)) \cap \pi_{cid}(\sigma_{pid='p02'}(O))$$

⑤ 既没有购买过 'p01'，也没有购买过 'p02' 号商品：

$$\pi_{cid}(C) - (\pi_{cid}(\sigma_{pid='p01'}(O)) \cup \pi_{cid}(\sigma_{pid='p02'}(O)))$$

⑥ 只购买过 'p01' 和 'p02' 号两种商品：

$$(\pi_{cid}(\sigma_{pid='p01'}(O)) \cap \pi_{cid}(\sigma_{pid='p02'}(O))) - \pi_{cid}(\sigma_{pid \neq 'p01' \wedge pid \neq 'p02'}(O))$$

□ 查询满足下述条件的顾客的编号：

⑦ 没有购买过商品： $\pi_{cid}(C) - \pi_{cid}(O)$

⑧ 只购买过一次商品（只有一份订单）：

令 $R := O$ ，该查询可表示如下：

$$\pi_{cid}(O) - \pi_{O.cid}(\sigma_{R.cid = O.cid \wedge R.ordno < O.ordno}(R \times O))$$

⑨ 只购买过同一种商品：

$$\pi_{cid}(O) - \pi_{O.cid}(\sigma_{R.cid = O.cid \wedge R.pid \neq O.pid}(R \times O))$$

关系代数的基本运算（总结）

运算	运算前提	结果关系模式	结果关系
$R \cup S$	R和S必须是同类关系	与关系R（或关系S）的模式相同	$\{t \mid t \in R \vee t \in S\}$
$R - S$			$\{t \mid t \in R \wedge t \notin S\}$
$R \times S$	任意两个关系	由关系R和关系S中的所有属性组成	$\{(t_r, t_s) \mid t_r \in R \wedge t_s \in S\}$
$\sigma_F(R)$	选择条件F是一个只能由关系R中的属性名、常量、比较运算符、逻辑运算符组成的逻辑表达式	与关系R的模式相同	$\{t \mid t \in R \wedge F(t) = true\}$
$\pi_A(R)$	属性组A只能是关系R的一个属性子集	由属性组A构成结果关系的关系模式	$\{t[A] \mid t \in R\}$

➤ **同类关系**：两个关系的关系模式同时满足以下两个要求

- ①具有相同的目n（具有相同数目的属性） ②相应的属性来自于同一个域

扩充运算 (小结)

□ 交运算: $R \cap S = \{ t \mid t \in R \wedge t \in S \}$

□ θ -连接: $R \bowtie_{A \theta B} S = \{ (t_r, t_s) \mid t_r \in R \wedge t_s \in S \wedge (t_r[A] \theta t_s[B]) \}$

□ 自然连接:

$$R \bowtie S = \{ (t_r[A], t_r[B], t_s[C]) \mid t_r \in R \wedge t_s \in S \wedge (t_r[B] = t_s[B]) \}$$

➤ 其中: 关系R的属性被划分为A和B两个属性组, 关系S的属性被划分为B和C两个属性组, 属性组B为两者之间的同名公共属性

□ 除运算: 给定关系 $R(X, Y)$ 和 $S(Y, Z)$, 其中 X, Y, Z 为属性组。

$$R \div S = \{ t_r[X] \mid t_r \in R \wedge \pi_Y(S) \subseteq Y_x \}$$

其中: Y_x 是 x 在 R 中的象集, $x = t_r[X]$

扩充运算的推导公式 (小结)

□ 交运算: $R \cap S = R - (R - S) = S - (S - R)$

□ θ -连接: $R \bowtie_{A \theta B} S = \sigma_{A \theta B}(R \times S)$

□ 自然连接运算的推导公式 $R \bowtie S$

➤ 设关系 R 和 S 的关系模式是:

- $Head(R) = \{A_1, A_2, \dots, A_n, C_1, C_2, \dots, C_k\}$
- $Head(S) = \{B_1, B_2, \dots, B_m, C_1, C_2, \dots, C_k\}$
- 它们之间的公共属性: $Head(R) \cap Head(S) = \{C_1, C_2, \dots, C_k\}$

➤ 结果关系的关系模式是:

$$Head(R \bowtie S) = \{A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m, C_1, C_2, \dots, C_k\}$$

➤ 结果关系的推导公式是:

$$R \bowtie S = \pi_{R.A_1, R.A_2, \dots, R.A_n, S.B_1, S.B_2, \dots, S.B_m, R.C_1, R.C_2, \dots, R.C_k} \left(\sigma_{R.C_1=S.C_1 \wedge R.C_2=S.C_2 \wedge \dots \wedge R.C_k=S.C_k} (R \times S) \right)$$

□ 除运算: 关系 $R(A_1, \dots, A_n, B_1, \dots, B_m)$ 和关系 $S(B_1, \dots, B_m)$

$$R \div S = \pi_{A_1, \dots, A_n}(R) - \pi_{A_1, \dots, A_n}((\pi_{A_1, \dots, A_n}(R) \times S) - R)$$